Programming Worlds

PROGRAMMING WORLDS ACADEMY

Git & GitHub 101

From Zero to Your First Pull Request

BEYOND CODE. BEYOND LIMITS.

11

MODULE 11

Professional Habits

Write a README, ignore files you should not track, and keep secrets safe.

MODULE 11 · LESSON 11.1

Writing a README with Markdown

كتابة README بـ Markdown

Learning objectives

- Write Markdown basics.
- Create a useful README.md.
- Preview it in VS Code.

Prerequisites

Module 5.

Introduction

The README is the front door of your repository. GitHub shows it automatically.

Explanation

Markdown is plain text with simple symbols. `#` makes a heading, `##` a smaller one, `-` a bullet, `1.` a numbered item, `**bold**` bold text, backticks inline code, three backticks a code block, `[text](url)` a link.

A good README answers: what is this, how do I run it, and who made it. Figure 1 shows a short example.

Create `README.md` in the repository root. In VS Code press `Ctrl+Shift+V` to preview it.

Commit it: `git add README.md`, `git commit -m "Improve README"`, `git push`.

On GitHub the README appears under the file list on the repository page.

Keep it short, accurate and updated.

Real-world analogy

> The README is the sign above a shop door: it tells visitors what is inside before they walk in.

Diagram

```
# My First Project

A small website I use to learn Git and GitHub.

## Features
- Home page
- About page

## How to run
1. Clone the repository
2. Open `index.html` in your browser

## Author
[Your name](https://github.com/YOUR-USERNAME)
```

Figure 1 · Example README.md (plain text)

Commands

```
git add README.md
git commit -m "Improve README"
git push
```

Figure 1 shows a small README.

Common mistakes and solutions

A blank README.	Add at least a title and a description.
Broken formatting from missing blank lines.	Leave a blank line between blocks.
Outdated instructions.	Update it when the project changes.

Practical exercise

> Write your own README with a title, features, run instructions and author.

Expected result

GitHub shows formatted headings and a link.

Knowledge check

1. What does # make in Markdown?

- ○ A link
- ○ A heading
- ○ A comment

2. What shortcut previews Markdown in VS Code?

- ○ Ctrl+Shift+V
- ○ Ctrl+Z
- ○ Alt+F4

3. Where is README.md placed?

- ○ Only on GitHub
- ○ Repository root
- ○ Anywhere in Windows

Answers

- A heading — More # means smaller.
- Ctrl+Shift+V — It opens the preview.
- Repository root — GitHub shows the root README.

Summary

A README explains what, how and who, written in Markdown.

Key vocabulary

Markdown	Text formatting language
README	Project introduction file
Preview	Rendered view of Markdown

Next lesson

Lesson 11.2 — .gitignore.

MODULE 11 · LESSON 11.2

Ignoring files with .gitignore

تجاهل الملفات بـ .gitignore

Learning objectives

- Explain .gitignore.
- Write patterns.
- Stop tracking a file already committed.

Prerequisites

Lesson 11.1.

Introduction

Some files should never be in Git: secrets, logs, system files and downloaded dependencies.

Explanation

`.gitignore` is a text file in the repository root. Each line is a pattern for files Git should ignore (Figure 1).

Common patterns: `.env`, `*.log` (any log file), `node_modules/` (a folder), `Thumbs.db`, `.DS_Store`.

Lines starting with `#` are comments. A leading `!` re-includes a file.

Create the file, commit it, and share it with the team so everyone ignores the same things.

Important: .gitignore only affects files Git is not tracking yet. If you already committed a file, run `git rm --cached FILE`, then commit. The file stays on your disk but leaves future commits.

If it was a secret, ignoring it is not enough; it is still in history (Lesson 11.3).

Check with `git status` that ignored files no longer appear.

Real-world analogy

> .gitignore is a “do not photograph” list for a camera that otherwise photographs everything.

Diagram

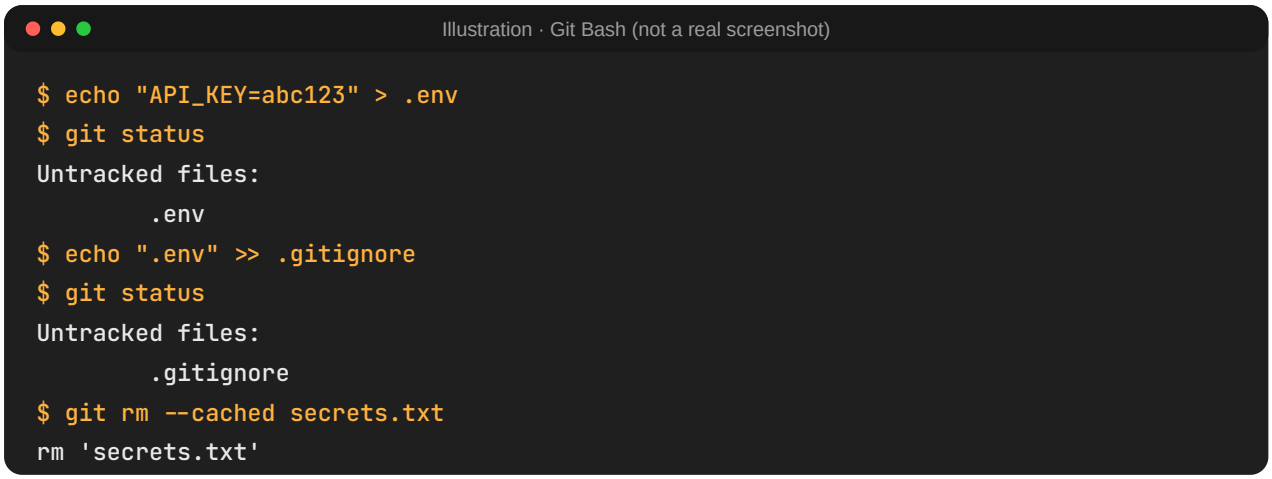


Figure 1 · Illustration: ignoring .env

Commands

`git rm --cached FILE`
`git status`

Figure 1 shows a secret file being ignored.

Common mistakes and solutions

Expecting .gitignore to untrack committed files.	Use git rm –cached.
Wrong file name such as gitignore.txt.	It must be `.gitignore` with no extension.
Ignoring too much.	Keep patterns specific.

Practical exercise

> Create .env with a fake value, ignore it, and confirm status does not list it.

Expected result

`git status` shows .gitignore but not .env.

Knowledge check

1. Where does .gitignore live?

- ○ GitHub only
- ○ Repository root
- ○ Desktop

2. Does .gitignore untrack committed files?

- ○ No
- ○ Yes
- ○ Only .env

3. What does *.log match?

- ○ Folders
- ○ All .log files
- ○ Only one file

Answers

- Repository root — Usually the root.
- No — Use git rm –cached.
- All .log files — * is a wildcard.

Summary

List patterns in .gitignore; use git rm –cached for files already tracked.

Key vocabulary

.gitignore	List of ignored patterns
pattern	Rule matching file names
git rm –cached	Untrack but keep on disk

Next lesson

Lesson 11.3 — Security and good habits.

MODULE 11 · LESSON 11.3

Security and good habits

الأمان والعادات الجيدة

Learning objectives

- Keep secrets out of Git.
- React to a leaked token.
- Adopt commit and branch habits.

Prerequisites

Lesson 11.2.

Introduction

Git remembers everything, so mistakes with secrets live forever unless you act.

Explanation

Never commit passwords, API keys, access tokens, private keys or `.env` files.

If you leaked one: first **revoke or rotate** it at the provider (for a GitHub token: Settings → Developer settings → Personal access tokens). Removing it from the latest commit is not enough, because history still has it. Treat it as public.

Turn on **two-factor authentication** on GitHub (Settings → Password and authentication).

This course uses HTTPS with Git Credential Manager, so you never store a password in a file.

Commit messages: short, imperative, specific — "Fix navigation spacing", not "changes".

Habits: pull before you start, commit small, one purpose per branch, review the diff before committing, never force-push shared branches, and keep main working.

Private repositories hide code from the public, but do not rely on them to protect secrets.

Real-world analogy

> A leaked key is a house key copied: changing the lock (rotating) is the real fix, not hiding the copy.

Diagram

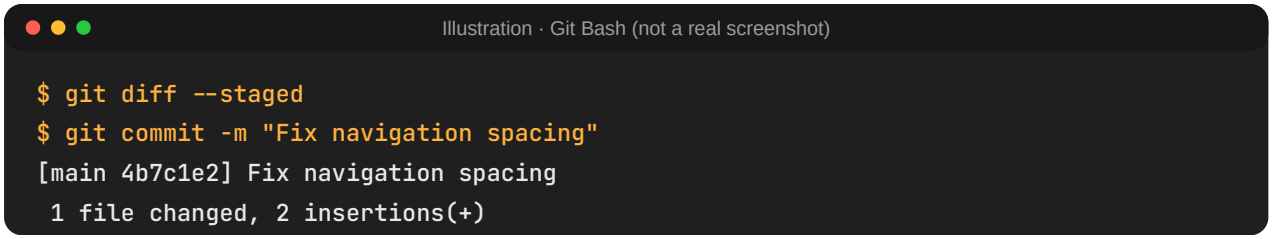


Figure 1 · Illustration: review before committing

Commands

No new commands. Revisit `git diff` and `git status` before each commit.

Common mistakes and solutions

Deleting the secret and calling it done.	Revoke it.
Using “update” as every message.	Say what changed.
Force-pushing main.	Avoid it; use revert.

Practical exercise

> Write five good commit messages for changes you have made in this course.

Expected result

Each message is under 72 characters and describes one change.

Knowledge check

1. First action when a token leaks?

- ○ Ignore it
- ○ Revoke or rotate it
- ○ Delete the repo only

2. Which is a good commit message?

- ○ Fix navigation spacing
- ○ stuff
- ○ asdf

3. What should 2FA protect?

- ○ Windows wallpaper
- ○ Your GitHub account
- ○ Your monitor

Answers

- Revoke or rotate it — History keeps it.
- Fix navigation spacing — Be specific.
- Your GitHub account — It blocks stolen passwords.

Summary

Keep secrets out, rotate leaks, enable 2FA, and keep commits clear and small.

Key vocabulary

Token	Secret used to authenticate
2FA	Second proof of identity
Rotate	Replace a secret with a new one

Next lesson

Module 12 — Final project.

MODULE 11

Practical lab 11 · README, .gitignore and safety

1. Create a branch: `git switch -c docs/readme`.
2. Rewrite README.md with a title, description, features, how to run and author link.
3. Create `.gitignore` with `.env`, `*.log`, `Thumbs.db`.
4. Create `.env` with `TEST=1` and check that `git status` does not list it.
5. Commit with clear messages, push, and open a pull request. Merge it.

Check	Pass if…
README	Renders with headings and a link on GitHub
.gitignore	.env is not in the repository
Messages	Specific and short

Evaluation: 3 of 3 = pass.

Module 11 quiz (pass mark: 70%)

1. What is README.md for?

- ○ Storing passwords
- ○ Running Git
- ○ Introducing the project

2. Which file lists ignored patterns?

- ○ index.html
- ○ .gitignore
- ○ .env

3. You committed .env earlier. What helps?

- ○ git rm –cached .env then rotate secrets
- ○ Nothing
- ○ Rename it

4. A good commit message is…

- ○ Very long
- ○ Specific and short
- ○ Empty

5. What does 2FA add?

- ○ Faster push
- ○ More storage
- ○ A second proof of identity

Answers

- Introducing the project — It is the front door.
- .gitignore — It lives in the root.
- git rm –cached .env then rotate secrets — And rotate the secret.
- Specific and short — Say what changed.
- A second proof of identity — It protects your account.